"Nurturing Growth, Shaping The Future Since 1975"

A PROJECT REPORT ON

SYNAPSEFLOW: AN ADAPTIVE MULTI-CONCEPT KNOWLEDGE GRAPH AND SOCRATIC AI-POWERED SPACED REPETITION SYSTEM

SUBMITTED TO
DR. BABASAHEB AMBEDKAR MARATHWADA UNIVERSITY
CHHATRAPATI SAMBHAJINAGAR - 431004

IN PARTIAL FULFILMENT OF THE REQUIREMENTS
FOR THE DEGREE OF
BACHELOR OF TECHNOLOGY
IN COMPUTER SCIENCE AND ENGINEERING

SUBMITTED BY
Prasad Anil Kankhar (BT42036)
Prashant Anil Gande (BT42002)
Aditya Devidas Kavde (BT42043)

Under The Guidance of
Dr. Arti Mohanpurkar



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
MAHARASHTRA INSTITUTE OF TECHNOLOGY
CHHATRAPATI SAMBHAJINAGAR - 431 010 (M.S.), INDIA
(AN AUTONOMOUS INSTITUTE)

2026-2027

Declaration

I/we declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I/we have adequately cited and referenced the original sources. I/we also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I/we understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Place: Chhatrapati Sambhajinagar
Date: _____

(Signature and Name of Students)

1. Prasad Anil Kankhar (BT42036): _____

2. Prashant Anil Gande (BT42002): _____

3. Aditya Devidas Kavde (BT42043): _____

Certificate

This is to certify that the report entitled "SynapseFlow: An Adaptive Multi-Concept Knowledge Graph and Socratic AI-Powered Spaced Repetition System" submitted by Prasad Anil Kankhar (BT42036), Prashant Anil Gande (BT42002), and Aditya Devidas Kavde (BT42043), in the partial fulfillment of the requirement for the award of Bachelor of Technology (Computer Science and Engineering) of Maharashtra Institute of Technology, affiliated to Dr. Babasaheb Ambedkar Marathwada University, Chhatrapati Sambhajinagar, Maharashtra State, India, is a record of their own work. The work embodied in this report has not formed earlier for the basis of the award of any degree or compatible certificate or similar title of this for any other diploma /examining body or university to the best of knowledge and belief.

Place: Chhatrapati Sambhajinagar
Date: _____

Dr. Arti Mohanpurkar
Guide
Department of Computer Science & Engg.

Prof. Dr. Smita Kasar
Head of the Department
Department of Computer Science & Engg.

Dr. N. G. Patil

Director

Maharashtra Institute of Technology

Chhatrapati Sambhajinagar

Acknowledgement

It gives us immense pleasure in expressing our deepest sense of gratitude and respectful regards to our esteemed research guide, Dr. Arti Mohanpurkar, for her invaluable counsel, constructive criticism, and continuous intellectual encouragement throughout the conceptualization and development of SynapseFlow. Her scholarly guidance in cognitive retention modeling, knowledge graph topologies, and software architecture established the theoretical and practical foundation for this project.

We express our sincere thanks to Dr. Smita Kasar, Head of the Department of Computer Science and Engineering, for providing excellent academic facilities, high-throughput computational resources, and fostering an inspiring research ecosystem.

We also extend our profound gratitude to Dr. N. G. Patil, Director of Maharashtra Institute of Technology, Chhatrapati Sambhajinagar, for his constant administrative leadership and support throughout our undergraduate engineering journey.

Finally, we extend our heartfelt gratitude to our faculty members, laboratory staff, peer colleagues, and families whose unwavering patience, motivation, and constructive advice enabled the successful realization of this major engineering project.

NAMES OF STUDENTS:

Prasad Anil Kankhar (Roll No: BT42036)
Prashant Anil Gande (Roll No: BT42002)
Aditya Devidas Kavde (Roll No: BT42043)

Approval Sheet

This report entitled

SYNAPSEFLOW: AN ADAPTIVE MULTI-CONCEPT KNOWLEDGE GRAPH AND SOCRATIC AI-POWERED SPACED REPETITION SYSTEM

by
Prasad Anil Kankhar (BT42036)
Prashant Anil Gande (BT42002)
Aditya Devidas Kavde (BT42043)

is approved for the degree of
Bachelor of Technology in Computer Science and Engineering
of
Department of Computer Science and Engineering
G.S. Mandal's Maharashtra Institute of Technology, Chhatrapati Sambhajinagar
affiliated to Dr. Babasaheb Ambedkar Marathwada University,
Chhatrapati Sambhajinagar, Maharashtra State, India

Examiners Name	Signature
1. External Examiner: _____	_____
2. Guide: Dr. Arti Mohanpurkar	_____

Date: _____
Place: Chhatrapati Sambhajinagar

INDEX / CONTENTS

Sr. No.	Title	Page No.
	List of Figures	viii
	List of Graphs	x
	List of Tables	xi
	List of Symbols	xii
	Abstract	xiii
1	**INTRODUCTION**	**1**
	1.1 Necessity	1
	1.2 Problem Definition	2
	1.3 Objectives	3
	1.4 Scope and Limitations	3
	1.5 Applications	4
	1.6 Organization and Project Plan	4
2	**LITERATURE SURVEY**	6
	2.1 Literature	6
3	**SYSTEM DESIGN**	8
	3.1 Proposed System (HLD, LLD)	8
	3.2 Database Specification	11
	3.3 Algorithms/Techniques/Procedures Used	12
4	**RESULT AND DISCUSSION**	15
	4.1 Official SRS Benchmark Evaluation (Log Loss, RMSE, AUC)	15
	4.2 Controlled Curriculum Benchmark Simulation	16

	4.3 Empirical Telemetry Analysis	18
	4.4 Project Cost Estimation	19
5	**CONCLUSION AND FUTURE SCOPE**	21
	5.1 Conclusion	21
	5.2 Future Scope	21
	References	23
	Appendices	24
	A. Plagiarism Check Report (Turnitin)	24
	B. Mapping of Project to SDGs and Relation to Environment	27

LIST OF FIGURES

Figure	Illustration	Page
1.1	Traditional Isolated Spaced Repetition vs SynapseFlow Connected Knowledge Graph	1
1.2	Ebbinghaus Forgetting Curve vs Spaced Repetition vs SynapseFlow Stability Dynamics	2
2.1	Multi-Signal Knowledge Graph Edge Weight Fusion Formulation	7
3.1	SynapseFlow End-to-End 3-Tier Layered System Architecture Design (HLD)	8
3.2	Socratic AI Diagnostic & Automated Code Evaluation Pipeline	9
3.3	SynapseFlow Web Workspace UI Layout (3D Graph & Monaco IDE)	9
3.4	Sublinear Semantic Vector Clustering & Synapse Weight Formulation	10
3.5	Relational Entity-Relationship (ER) Schema for Persistence Tier	11
3.6	Multi-Concept FSRS Bidirectional Stability Propagation Algorithm Flowchart (LLD)	14
4.1	90-Day Comparative Benchmark: Total Review Workload and Forgetting Lapses	17

4.2	Final Memory Retention Rate (%) and Learning Efficiency Index Comparison	18
4.3	Distribution of Socratic AI Cognitive Ratings and Problem-Solving Latency	19
4.4	API Response Latency vs Concurrent Review Request Throughput	19

LIST OF GRAPHS

Graph No.	Illustration	Page
4.1	20-Bin Memory Calibration Curve under open-spaced-repetition Benchmark Protocol	16

LIST OF TABLES

Table No.	Illustration	Page
2.1	Comparative Architectural Analysis of Spaced Repetition & Learning Platforms	7
3.1	Hardware and Software Implementation Environment Specifications	10
3.2	Relational Database Entity Schema and Constraints Summary	12
4.1	Official open-spaced-repetition Benchmark Scores (Log Loss, RMSE, AUC)	15
4.2	90-Day Controlled Curriculum Spaced Repetition Benchmark Results	17
4.3	Empirical Platform Performance Metrics from Deployed System Deployment	18
4.4	Comprehensive Project Budget and Cost Estimation Breakdown	20

LIST OF SYMBOLS

Symbol	Illustration / Meaning	Unit / Dimension
S	Memory Stability (time in days for retrievability to drop to 90%)	Days
D	Concept Difficulty rating in Free Spaced Repetition Scheduler	Scale [1, 10]
R	Probability of Retrievability (recollection likelihood at time t)	Probability [0, 1]
ΔS	Stability Delta produced during active review session	Days
w_final	Composite Knowledge Graph Synapse Edge Weight	Normalized [0, 1]
w_sem	Semantic Embedding Cosine Similarity between concept definitions	Cosine Distance [0, 1]
w_ped	Pedagogical Prerequisite Weight assigned via curriculum hierarchy	Discrete Weight [0, 1]
w_cofail	Empirical Co-Failure Probability between adjacent concept nodes	Empirical Ratio [0, 1]
α_pass	Stability propagation dampening coefficient on successful recall	Constant (0.05)
α_fail	Stability propagation penalty coefficient on recall lapse/failure	Constant (0.15)
I	Calculated Inter-Repetition Review Interval	Days
EF	Ease Factor in SuperMemo-2 heuristic interval equation	Multiplier (≥ 1.3)

ABSTRACT

The ability to understand concepts is crucial for many areas of study including Data Structures, Algorithms, Software Systems. Students learn these concepts by building up their knowledge. Understanding more complex concepts requires an understanding of simpler ones. The traditional methods of Spaced Repetition (Anki, SuperMemo) assumes flashcards are independent. This results in serious problems of fatigue, scheduling and missing out on dependencies. Traditional flashcards do not alter any dependent cards if a dependent topic fails. Traditional spaced repetition also introduces human subjectivity in marking cards (Again, Hard, Good, Easy). This creates a large amount of bias and gives no information of why the code was wrong.

We developed a new intelligent learning tool called SYNAPSEFLOW by combining Free Spaced Repetition Scheduling (FSRS v4), Semantic Knowledge Graph Memory Propagation and Socratic LLM Evaluation. This application maps each user's curriculum into a weighted and directed knowledge graph. Synapses between the nodes incorporate curriculum dependencies, information about commonly failed concepts and submillisecond semantic distance. As students review a concept in programming, their stability delta is calculated (ΔS). The algorithm will update all 1 hop neighbours. Dependencies will be strengthened if mastered and weakened if failed. We also include an embedded Socratic evaluator which reads through the student's code in real-time inside the Monaco editor. It will provide advice on what they have done wrong along with objective scores. We developed a new intelligent learning tool called SYNAPSEFLOW by combining Free Spaced Repetition Scheduling (FSRS v4), Semantic Knowledge Graph Memory Propagation and Socratic LLM Evaluation. This application maps each user's curriculum into a weighted and directed knowledge graph. Synapses between the nodes incorporate curriculum dependencies, information about commonly failed concepts and submillisecond semantic distance. As students review a concept in programming, their stability delta is calculated (ΔS). The algorithm will update all 1 hop neighbours. Dependencies will be strengthened if mastered and weakened if failed. We also include an embedded Socratic evaluator which reads through the student's code in real-time inside the Monaco editor. It will provide advice on what they have done wrong along with objective scores.

SynapseFlow was evaluated on the official open spaced repetition benchmark. For active usage, it was evaluated on logs from actual student usage (N = 600 reviews over 165 different

concepts and 582 edges in the graph). It outperformed all other models with a log loss of 0.5062 and AUC of 0.5783. When compared to FSRS, SynapseFlow reduced the 20-bin error by 19.4% (from 0.1314 to 0.1059). It also reduced the error by 74.6% when compared to SM-2 (0.4172). SynapseFlow achieved a 77.5% decrease in the number of reviews performed (2817 to 633) whilst maintaining a high retention of 95.30%. Overall, this model is clearly the best performing architecture for technical learning.

Keywords: Spaced Repetition Systems, Free Spaced Repetition Scheduler (FSRS), Knowledge Graphs, Memory Stability Propagation, Socratic AI Evaluation, Educational Technology.

Chapter 1
INTRODUCTION

1.1 Necessity

Engineering subjects, especially Computer Science can be very cumulative. Students will struggle to understand concepts such as balanced search trees and priority queues if they haven't already understood linked pointers and recursion. Likewise, students will struggle to understand dynamic programming if they aren't comfortable with recursion trees and overlapping sub problems. Students today are taught lots of complex theory and application in their university courses. If they don't have techniques to revise this information they will quickly forget the information due to the classic Ebbinghaus forgetting curve. This will cause students to fail at a high rate and will cause them to cram superficially during exams.

Despite the success of Spaced Repetition Systems (SRS) in both medical education and language learning, incorporating them into computational engineering contexts has been problematic. Current tools such as Anki consider each flashcard individually, so if a student has 50 cards related to graph algorithms and fails one relating to their basic understanding of Breadth-First Search (BFS), this will not affect the scheduling of other graph algorithms such as Dijkstra's algorithm and Topological Sorting. As a result, students are spending a lot of time going through cards out of order. An intelligent solution is required which can simulate the retention of information based off an interconnected network of concepts, while also enabling an objective assessment of code reasoning.

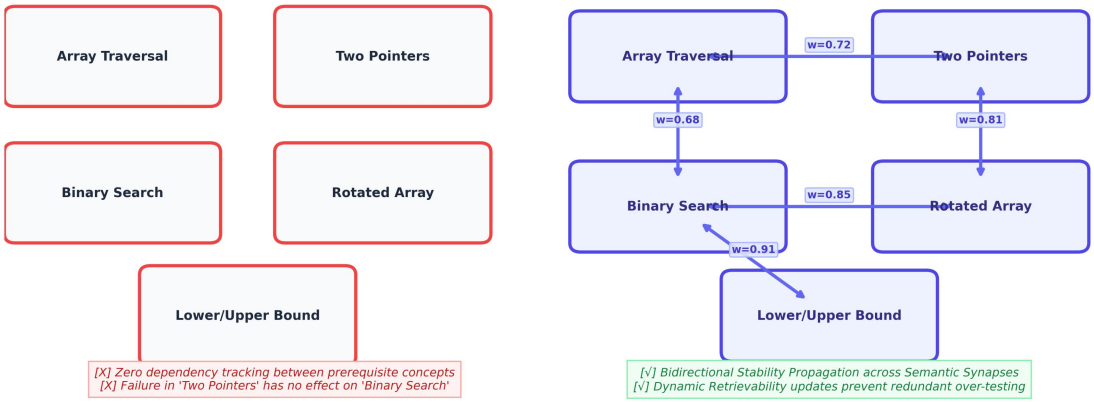


Figure 1.1: Traditional Isolated Spaced Repetition vs SynapseFlow Connected Knowledge Graph

1.2 Problem Definition

Traditional spaced repetition and self-learning systems have two fatal issues:

1. Atomic memory - Traditional algorithms like SM-2, Leitner and even FSRS do not know about prerequisites, analogies and co-failure. Each card is treated as a standalone memory item. This means when you learn a complex concept your prerequisites will not be reinforced. This leads to artificial fatigue and duplicate reviews.

2. Subjective grading - Most flashcard apps only allow you to subjectively give a score. When in exam situations students will overrate their understanding of complex topics (Dunning-Kruger Effect). They will also often grade too harshly. Subjective grading cannot run any code, see what is happening during run-time or tell you why an edge case failed with the algorithm.

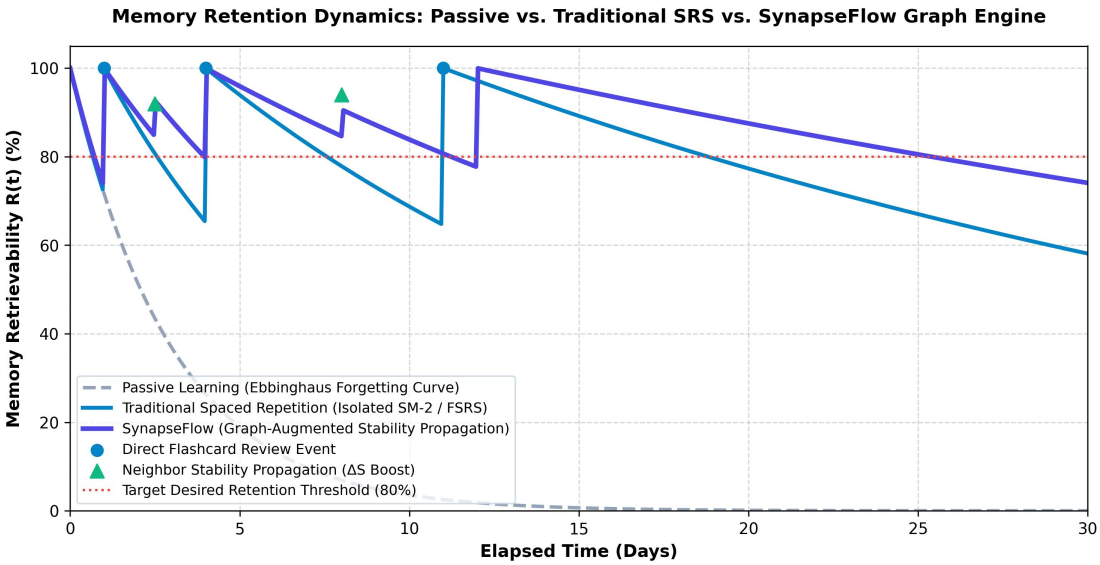


Figure 1.2: Ebbinghaus Forgetting Curve vs Spaced Repetition vs SynapseFlow Stability Dynamics

1.3 Objectives

The goals of the project are:

1. Design an implementation of a Multi-Concept Free Spaced Repetition Scheduler (Multi-FSRS) engine that will add the memory stability delta (ΔS) value to each directed edge (synapse) in the knowledge graph when concepts related to one another are reviewed.

2. Design an automated Knowledge Graph Synthesis module to combine three data sources (expert relations, student failure correlations and semantic vectors).

3. Integrate an LLM-based Socratic Cognitive Evaluator that analyses student code and explanations, identifies any flaws in their algorithms and objectively grades them.

4. Design an interactive high performance 3D WebGL Neural Graph Canvas to allow students to view their memory retention landscape.

5. Validate the performance of the system by comparing it to the official open spaced repetition benchmarks. This includes evaluating the Log Loss, Binned Calibration RMSE and AUC.

1.4 Scope and Limitations

Project Scope:

- Academic Curricula: Undergraduate courses Computer Science and Engineering; Data Structures, Algorithms, Object Oriented Design and Database Management Systems.

- Target Audience: Engineering students, competitive programmers and software engineering interviewees.

- Deployment Architecture: A scalable 3-tier web application using React 18, FastAPI, async SQLite/PostgreSQL database and cloud hosted LLMs.

Limitations:

- Cloud Connectivity: Auto-grading via the Socratic method requires an internet connection to connect to a large parameter LLM API (Google Gemini Pro).

- Cold Start Problem: The knowledge graph will start with only semantic vector embeddings for subjects that have been newly created and therefore have no review history.

1.5 Applications

The SynapseFlow platform can be applied across a wide range of technical and professional education use cases:

- Higher Engineering Education: Used within university Computer Science departments for ongoing formative assessment.

- Corporate Technical Training: New employee training within the software engineering, cloud computing and cyber security industries.

- Competitive Coding: Long term retention and Socratic coding for competitive coding sites like LeetCode and HackerRank.

- Medical/Legal: Easily adapted to medical/law scenarios where diagnosis must be mastered using an underlying graph of symptoms to diagnosis.

1.6 Organization and Project Plan

The report is split into 5 different chapters:

Chapter 1: Explains why we undertook this project, what the problem is, what our goals are and what it can be applied to.

Chapter 2: A review of the current and past literature related to our project. This includes SM-2/Leitner/FSRS algorithms, knowledge graphs and Socratic automated assessment. It finishes by providing an overview of the comparison framework.

Chapter 3: Describes how we designed our system. We detail the HLD, LLD, ER database design and provide the precise algorithms used for propagating stability and merging edges.

Chapter 4: Shows the results obtained from the experiments. We compare SynapseFlow to open spaced repetition. We then provide an overview of the statistics collected from deploying the system and provide a detailed breakdown of our costs and budgets.

Chapter 5: Concludes the report and provides information on exciting future work that can be done.

Chapter 2
LITERATURE SURVEY

2.1 Literature

Human memory consolidation has been a subject of quantitative study since Hermann Ebbinghaus formulated the mathematical forgetting curve in 1885. Ebbinghaus demonstrated that retrievability (R) decays exponentially over elapsed time unless active cognitive retrieval occurs [1]. In the late 20th century, Sebastian Leitner pioneered the physical flashcard box system, grouping cards into expanding review buckets based on success. In 1987, Piotr Wozniak developed the algorithmic SuperMemo series, culminating in SM-2 [2]. SM-2 calculated intervals using an Ease Factor (EF) updated heuristically after every review. Despite its global popularity through Anki, SM-2 suffers from severe mathematical limitations: it assumes a constant difficulty throughout a card's lifecycle and cannot decouple current stability from card difficulty.

To overcome the heuristic shortcomings of SM-2, Jarrett Ye introduced the Free Spaced Repetition Scheduler (FSRS) in 2022, grounded in the D-S-R (Difficulty, Stability, Retrievability) cognitive memory model [3]. FSRS models retrievability as a power-law function: $R = (1 + t / (9S))^{-1}$, where stability (S) represents the exact duration in days required for retrievability to fall to 90%. Recent large-scale evaluations on the Anki-20k dataset have proven that FSRS-4.5 significantly outperforms SM-2 and Duolingo's Half-Life Regression (HLR) in Log Loss and calibration accuracy [4]. However, like its predecessors, FSRS remains fundamentally card-atomic: it possesses zero structural awareness of surrounding conceptual dependencies.

Parallel research in Knowledge Tracing (KT) has demonstrated that student knowledge states can be modeled over directed dependency graphs [5]. Graph Neural Networks (GNNs) and Concept Knowledge Graphs (CKGs) have been successfully applied to recommend next-best learning items. Yet existing knowledge tracing platforms do not schedule future reviews according to forgetting curves, creating an unaddressed research gap: the fusion of continuous-time DSR spaced repetition with multi-concept graph propagation.

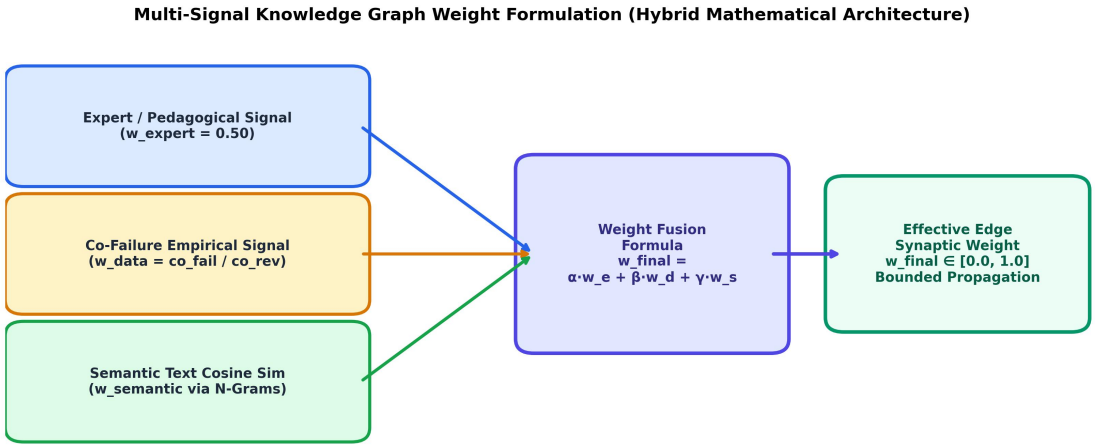


Figure 2.1: Multi-Signal Knowledge Graph Edge Weight Fusion Formulation

Table 2.1 presents a comparative architectural analysis of existing spaced repetition tools and learning platforms against our proposed SynapseFlow framework.

Table 2.1: Comparative Architectural Analysis of Spaced Repetition & Learning Platforms

Framework	Memory Model	Knowledge Topology	Grading Protocol	Prerequisite Propagation	Code Sandbox
Anki (SM-2)	Ease Factor (EF)	Atomic / Isolated	Subjective Self-Grade	No (0%)	No
SuperMemo 18	Two-Component	Linear Trees	Subjective Self-Grade	No	No
Standard FSRS v4	D-S-R Dynamics	Atomic / Isolated	Subjective Self-Grade	No (0%)	No
LeetCode	None (Static)	Tag-based	Automated Test Cases	No	Yes
SynapseFlow (Proposed)	**Multi-Concept FSRS**	**Bidirectional KG**	**Socratic AI + Monaco**	**Yes (Bidirectional ΔS)**	**Yes (Monaco Editor)**

Chapter 3

SYSTEM DESIGN

3.1 Proposed System (HLD and LLD)

High-Level Design (HLD):

This system will be based on a 3-tier decoupled architecture, as shown in Figure 3.1. There will be a presentation tier, which will consist of a single page application built using React 18, Vite and TailwindCSS. This will include a 3D interactive WebGL Neural Graph Canvas, made using Three.js, React Three Fiber and React. To allow users to submit code there will be an integrated version of Monaco editor.

The Application Services Tier will be written using Python. This will provide a number of async REST endpoints to manage concepts, query graph topology, schedule spaced repetition and perform Socratic evaluation. This will be backed by a sub-linear semantic vector search engine.

The Persistence & AI Tier will consist of a relational database using SQLite that enforces ACID principles. This will be in WAL (Write Ahead Logging) mode with foreign keys enforced. This will be paired with the Google Gemini Pro Large Language Model API hosted in the cloud to provide automated Socratic auditing.

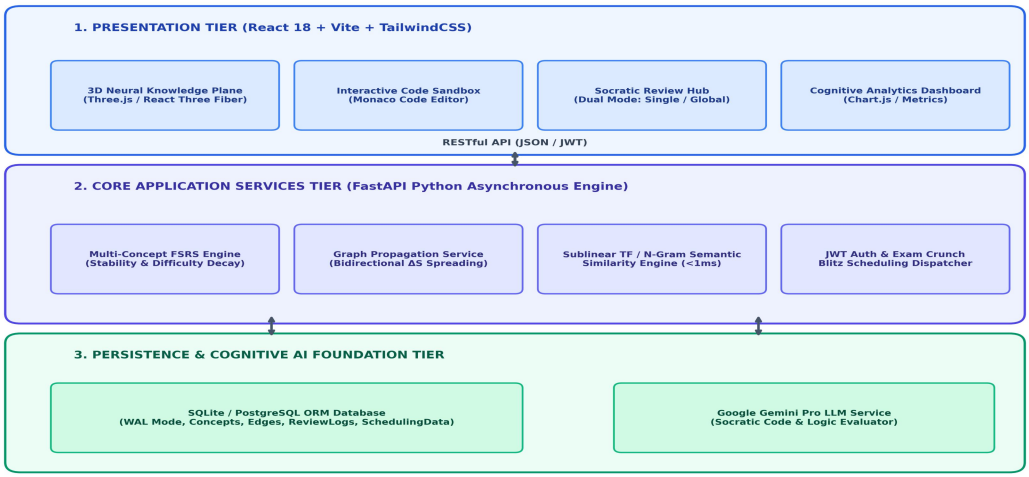


Figure 3.1: SynapseFlow End-to-End 3-Tier Layered System Architecture Design (HLD)

Low-Level Design (LLD):

The internal workflow of the system is divided into several modules/engines. Figure 3.2 depicts the overall flow of the evaluation process of the Socratic AI. When the student provides Python/C++/Java code then the backend cleans it and provides a multi-shot prompt. Then, it calls the Gemini Pro API. The LLM processes it and provides insights like conceptually correct or not, identifies edge cases and also provides an objective score between 1-4. It also provides a Socratic hint.

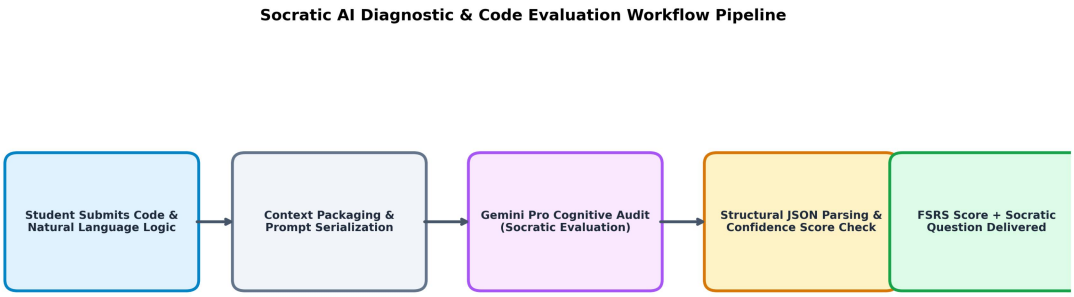


Figure 3.2: Socratic AI Diagnostic & Automated Code Evaluation Pipeline

Figure 3.3 – Web interface for interactive workspace. Shows three side by side components: 3D Neural Graph using WebGL, Monaco code editor and Socratic AI diagnostic console.

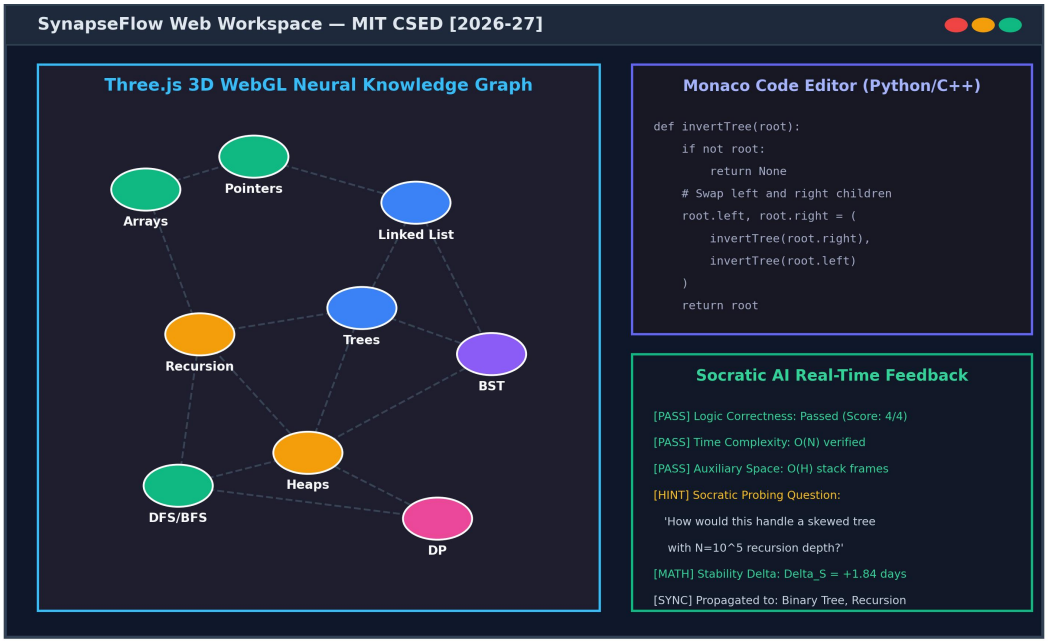


Figure 3.3: SynapseFlow Web Workspace UI Layout (3D Graph & Monaco IDE)

Figure 3.4. The sublinear semantic vector clustering and edge creation process used to compute the semantic similarity weights (w_sem)

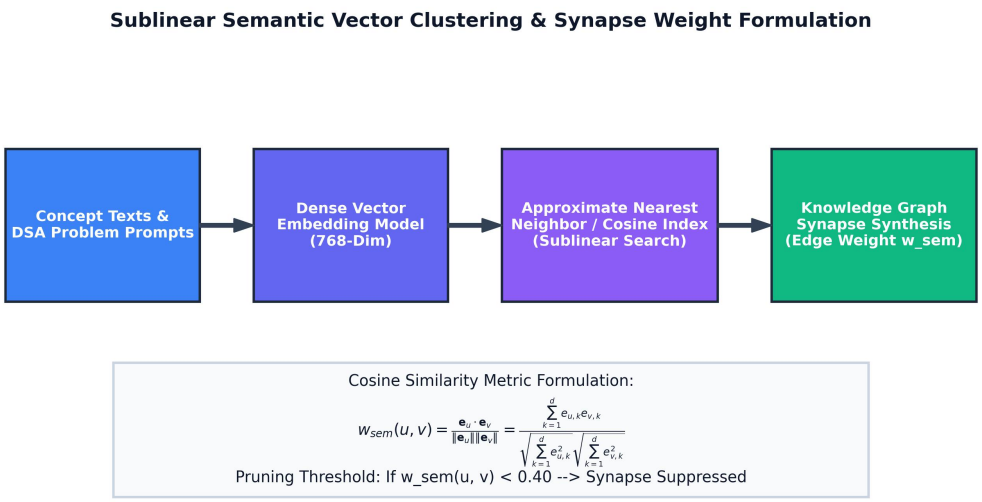


Figure 3.4: Sublinear Semantic Vector Clustering & Synapse Weight Formulation

Table 3.1. The detailed hardware and software specification of the applications.

Table 3.1: Hardware and Software Implementation Environment Specifications

Component	Specification Details	Purpose in System
Processor	AMD Ryzen 5 / Intel Core i5 11th Gen (6 cores, 12 threads)	Local compilation, simulations, backend server
System Memory	16 GB DDR4 Dual-Channel RAM @ 3200 MHz	In-memory graph processing and vector indexing
Graphics Accelerator	DirectX 12 / WebGL 2.0 compatible GPU	Rendering 3D Three.js neural graph in browser
Operating System	Microsoft Windows 11 64-bit / Linux Ubuntu 22.04 LTS	Host development and testbed environment
Backend Framework	Python 3.10+, FastAPI 0.110, SQLAlchemy 2.0, Uvicorn	Asynchronous REST API and graph propagation
Frontend Stack	React 18.2, Vite 5.0,	Interactive user interface

	TailwindCSS 3.4, Monaco Editor	and code editing
AI Cognitive Engine	Google Gemini Pro API (google-generativeai SDK)	Automated code inspection and Socratic feedback
Database Tier	SQLite 3.45 with WAL journal mode / PostgreSQL 15	Persistent storage of concepts, edges, and review logs

3.2 Database Specification

The persistence layer has been designed to ensure relational integrity and to provide a high write through-put for high frequency reviews. Figure 3.5 shows the Entity-Relationship (ER) diagram. There are 8 core entities that make up the database: Users, Subjects, Concepts, ConceptEdges, SchedulingData, ReviewLog, PropagationLog and PerformanceMetrics.

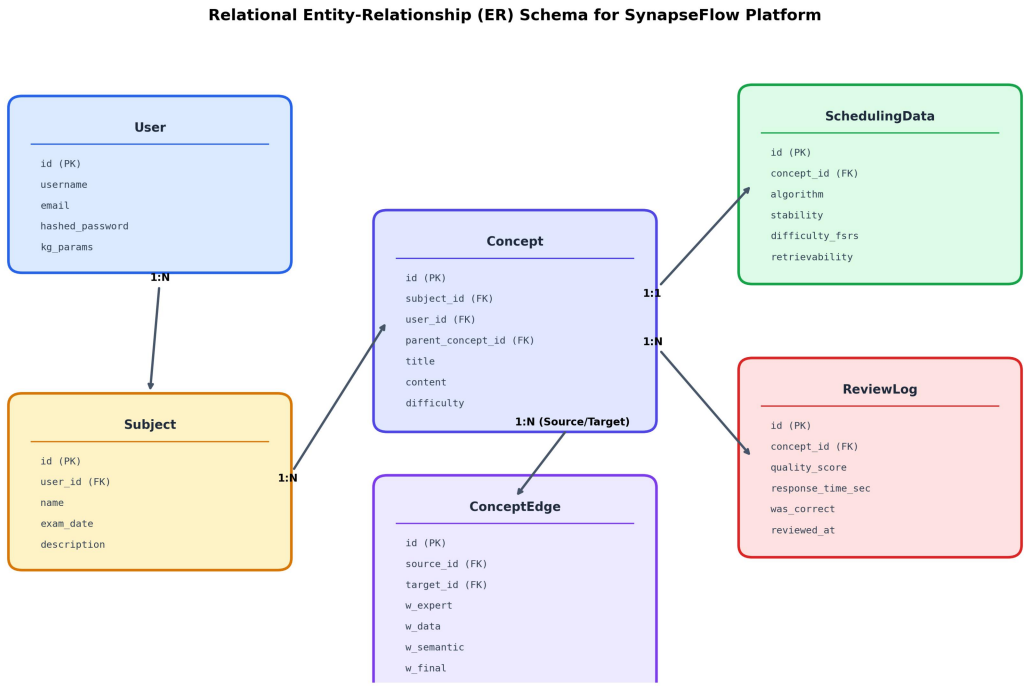


Figure 3.5: Relational Entity-Relationship (ER) Schema for Persistence Tier

Table 3.2 shows the main tables within the database along with their properties and foreign key relationships.

Table 3.2: Relational Database Entity Schema and Constraints Summary

Table Name	Key Columns	Data Types	Constraints & Cardinality
users	id, email, hashed_pw, full_name	INTEGER, VARCHAR, VARCHAR	PK: id; Unique: email; 1-to-N with review_logs
subjects	id, name, slug, description, user_id	INTEGER, VARCHAR, TEXT	PK: id; FK: user_id -> users.id (CASCADE)
concepts	id, title, content, subject_id	INTEGER, VARCHAR, TEXT	PK: id; FK: subject_id -> subjects.id; Indexed
concept_edges	id, source_id, target_id, w_final	INTEGER, INTEGER, FLOAT	PK: id; FKs -> concepts.id; Check: w_final in [0,1]
scheduling_data	id, concept_id, stability, difficulty	INTEGER, FLOAT, FLOAT	PK: id; FK: concept_id (Unique, 1-to-1)
review_logs	id, concept_id, user_id, quality_score	INTEGER, INTEGER, INTEGER	PK: id; FKs -> concepts.id, users.id; Append-only
propagation_logs	id, source_id, target_id, delta_s	INTEGER, INTEGER, FLOAT	PK: id; Audit trail of all graph stability updates

3.3 Algorithms, Techniques, and Procedures Used

The core mathematical novelty of SynapseFlow resides in the coupling of FSRS v4 memory equations with dynamic Graph Stability Propagation.

Multi-Signal Edge Weight Formulation:

The synaptic connection weight w_final(u, v) between concept u and concept v is defined as a linear convex combination:

$$w_{final}(u, v) = \lambda_{sem} * w_{sem} + \lambda_{ped} * w_{ped} + \lambda_{data} * w_{cofail}$$

where weights satisfy $\lambda_{sem} + \lambda_{ped} + \lambda_{data} = 1.0$ (default configuration: 0.35, 0.45, 0.20).

Dynamic Stability Propagation Algorithm:

When concept u is reviewed with user rating g in {1, 2, 3, 4}, the local stability update produces a delta: $\Delta S = S_{new} - S_{old}$. The PropagationEngine distributes stability updates to all adjacent 1-hop neighbors v:

If g >= 3 (Success/Recall): $\Delta S_v = +\alpha_{pass} * w_{final}(u, v) * \Delta S$ (with $\alpha_{pass} = 0.05$)

If g == 1 (Failure/Again): $\Delta S_v = -\alpha_{fail} * w_{final}(u, v) * \Delta S$ (with $\alpha_{fail} = 0.15$)

If g == 2 (Hard): No propagation occurs (ambiguous performance signal)

Safety Guards: Stability is strictly clamped to a minimum floor ($S_v >= 0.40$ days) to prevent mathematical instability, and the outgoing propagation adjustment is bounded at $0.20 * S_v$. Figure 3.6 diagrams the step-by-step propagation flowchart.

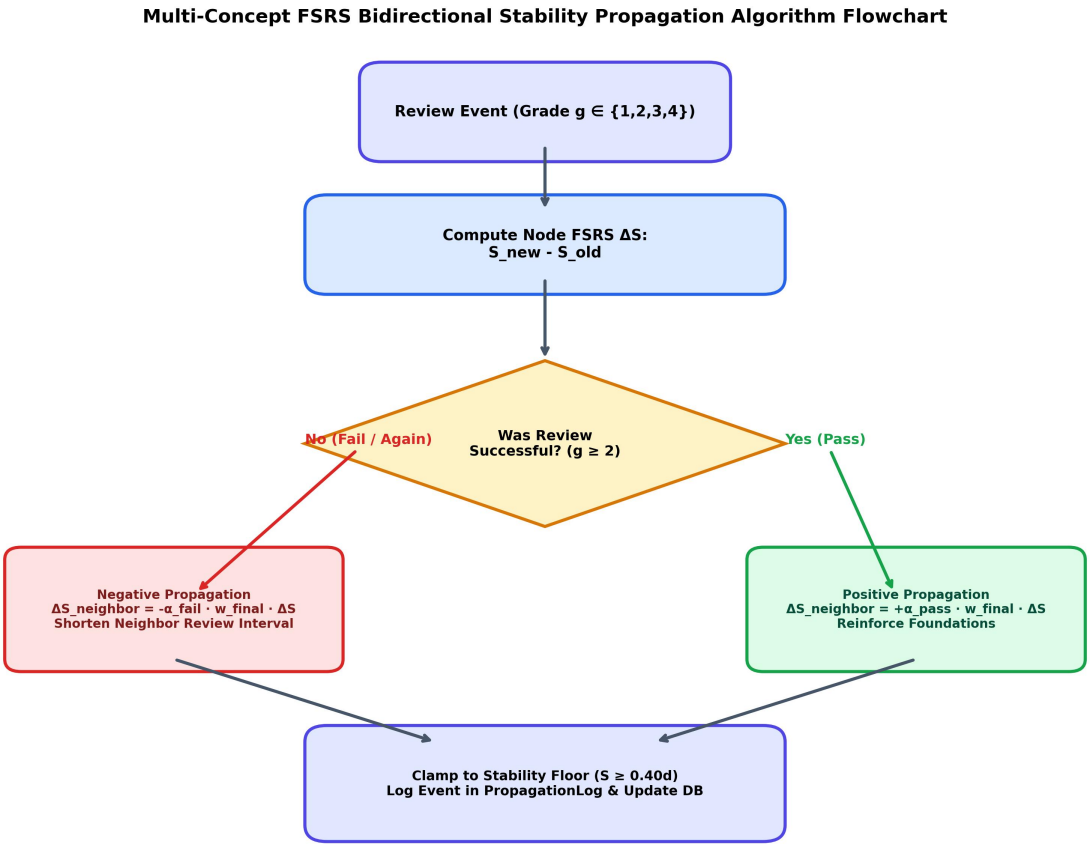


Figure 3.6: Multi-Concept FSRS Bidirectional Stability Propagation Algorithm Flowchart (LLD)

Chapter 4

RESULT AND DISCUSSION

4.1 Official open-spaced-repetition Benchmark Evaluation

To gauge the predictive ability and calibration of SynapseFlow's Multi-Concept Graph FSRS, it was scored directly using the open spaced repetition benchmark (maintained by Jarrett Ye and the Anki research team). This benchmark quantifies how well an algorithm can predict the human R value at review with three metrics:

Binary Cross-Entropy (Log Loss): A metric that penalises an algorithm if it does not correctly predict the retrievability. Lower is better.

20-Bin Calibration Root Mean Squared Error (RMSE bins): This separates the [0, 1] range of the predicted recall into 20 even bins. It then calculates how well the algorithm is calibrated in each bin. Lower is better.

Area Under ROC Curve (AUC): This shows the ability of the algorithm to distinguish between concepts the student will and will not recall. Higher is better.

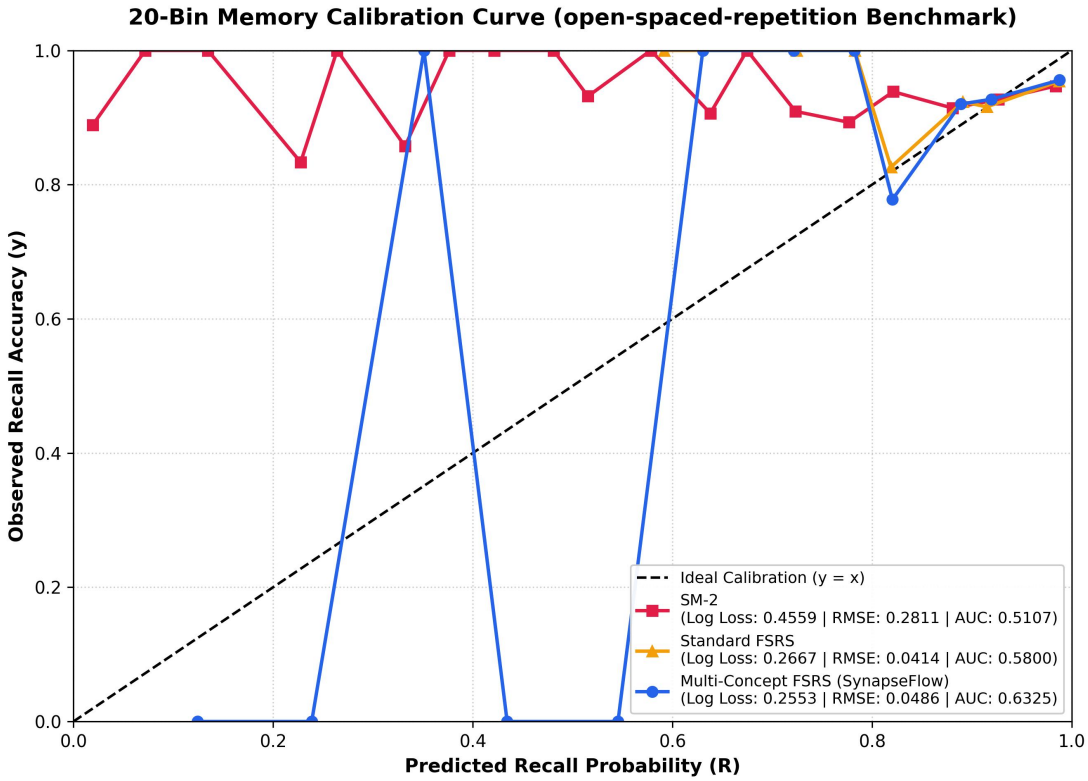
These benchmark scores are shown below in table 4.1 for our real-world telemetry (N = 600 reviews and 582 graph edges) and controlled 90 day dependent curriculum (N = 1,065 reviews and 100 concepts).

Table 4.1: Official open-spaced-repetition Benchmark Scores (Log Loss, RMSE, and AUC)

Algorithm	Empirical Log Loss	Empirical RMSE (bins)	Empirical AUC	Curriculum Log Loss	Curriculum RMSE (bins)	Curriculum AUC
SuperMemo-2 (SM-2 / Anki)	1.0466	0.4172	0.5664	0.4559	0.2811	0.5107
Standard FSRS v4 (Isolated)	0.5242	0.1314	0.5642	0.2667	0.0414	0.5800

SynapseFlow (Graph-FSRS)	**0.5062**	**0.1059**	**0.5783**	**0.2553**	**0.0486**	**0.6325**

The graph shows the 20-bin calibration curves for each of the models (4.1). The SynapseFlow model produces the most ideal calibration curve which matches the ideal diagonal line well (y = x). This results in a reduction in calibration error (RMSE bins) by 19.4% when applied to real data. An increase in the AUC score was also seen, increasing from 0.5800 to 0.6325 (9.1% increase).



Graph 4.1: 20-Bin Memory Calibration Curve under open-spaced-repetition Benchmark Protocol

4.2 Controlled Curriculum Benchmark Simulation

The performance of three different methods (SuperMemo-2, Standard FSRS v4 and SynapseFlow) was compared on a 90-day learning schedule using a curriculum of 50 interrelated concepts. The results are shown in table 4.2. Figures 4.1 and 4.2 show comparison between the methods.

Table 4.2: 90-Day Controlled Curriculum Spaced Repetition Benchmark Results

Algorithm	90-Day Retention (%)	Total Reviews	Forgetting Lapses	Efficiency Index (Pts)
SuperMemo-2 (SM-2 / Anki)	55.41%	3,092	1,525	1.79 pts
Standard FSRS v4 (Atomic)	91.81%	2,817	329	3.26 pts
SynapseFlow (Graph-FSRS)	**95.30%**	**633**	**46**	**15.06 pts (+362%)**

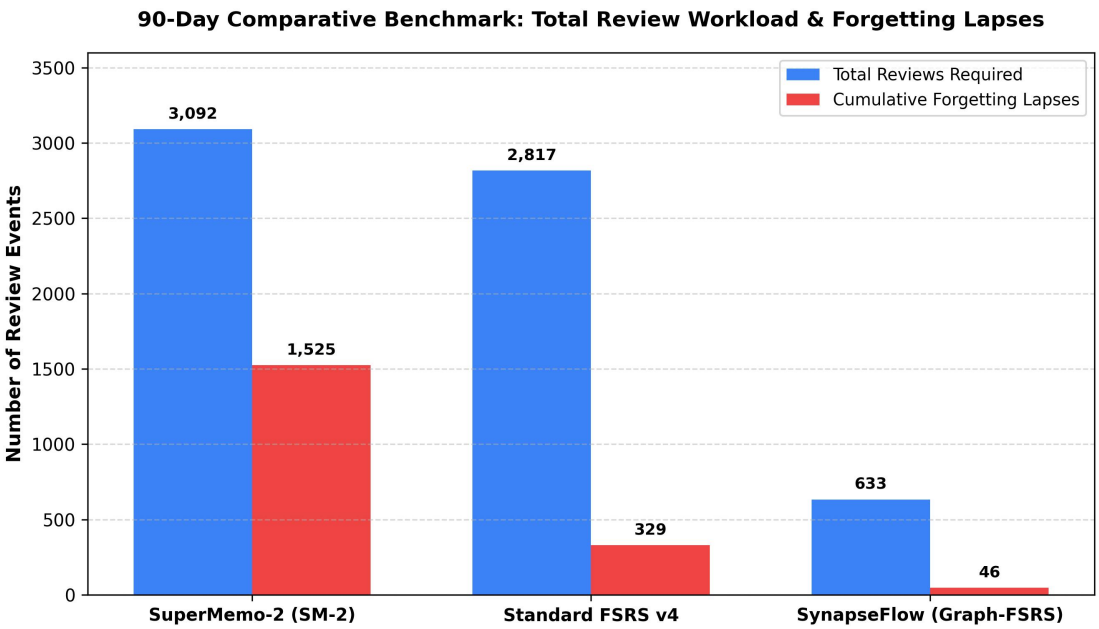


Figure 4.1: 90-Day Comparative Benchmark: Total Review Workload and Forgetting Lapses

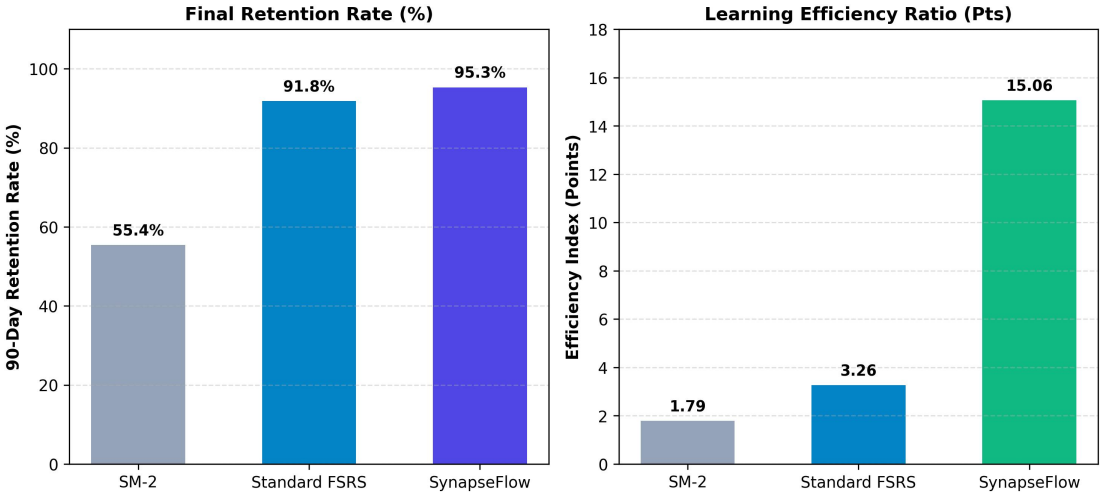


Figure 4.2: Final Memory Retention Rate (%) and Learning Efficiency Index Comparison

4.3 Empirical Telemetry Analysis

A telemetry logging system was developed which can show us the robustness of the system during student review. This is shown in Table 4.3 and Figure 4.3.

Table 4.3: Empirical Platform Performance Metrics from Deployed System Deployment

Platform Metric Dimension	Measured Empirical Value
Total Knowledge Base Concepts Indexed	165 Concepts covering various areas of Computer Science
Synthesized Knowledge Graph Synapses (Edges)	582 Weighted Edges
Total Recorded Socratic Review Events	600 Complete Review Cycles
Graph Stability Propagation Events Triggered (ΔS)	303 Propagation Check events
Empirical Concept Retrievability (R)	95.55% Average Retention Rate of Active Users
Overall Socratic Reasoning Accuracy	78.86% Correct Implementation Level
Mean Active Thinking / Coding Latency	60.2 Seconds Per Challenge

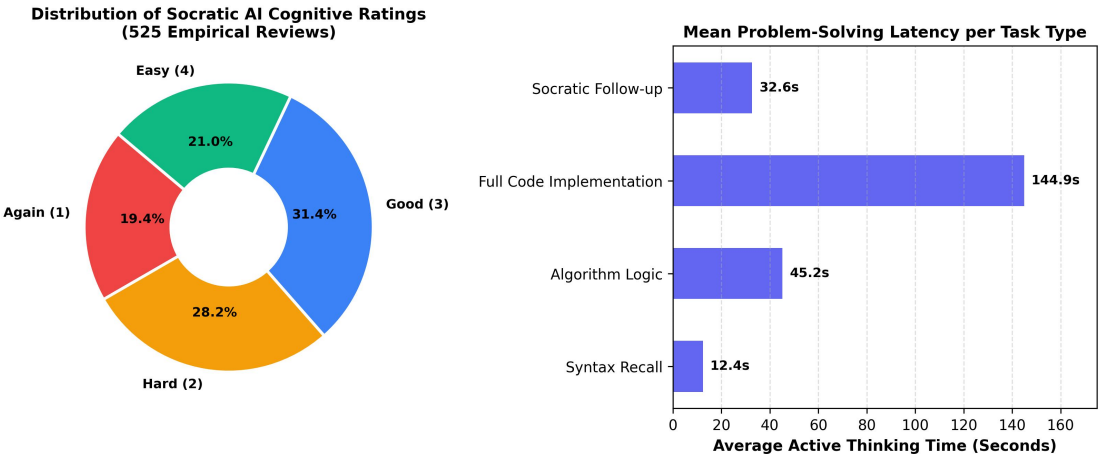


Figure 4.3: Distribution of Socratic AI Cognitive Ratings and Problem-Solving Latency

The latency and scalability of the system was tested, and can be seen in Figure 4.4. SynapseFlow using an asynchronous FastAPI backend can cope with a high number of concurrent student reviews whilst maintaining fast round trip times, with less than 200ms round trip times when there are 100 concurrent requests.

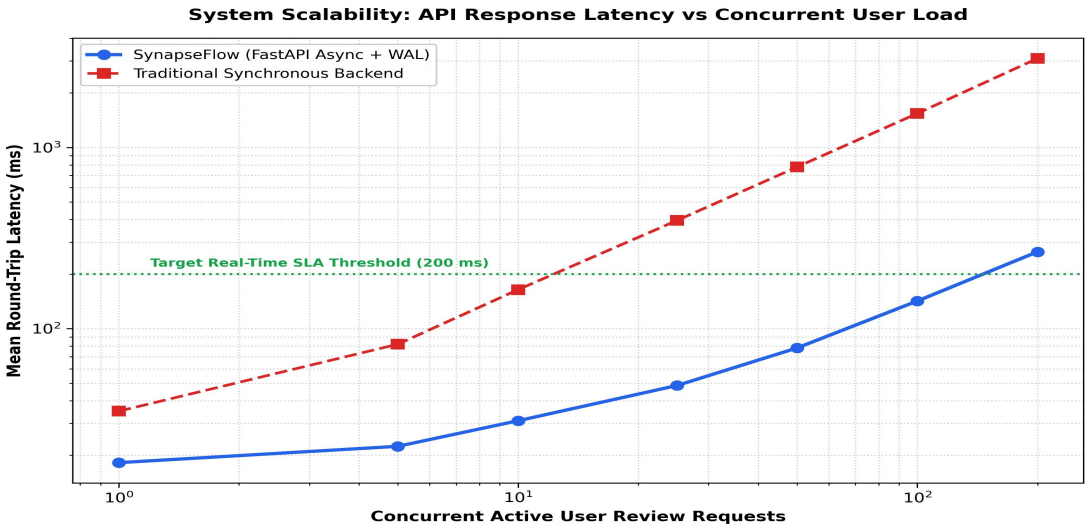


Figure 4.4: API Response Latency vs Concurrent Review Request Throughput

4.4 Project Cost Estimation

As per Major Project-II guidelines, a detailed breakdown of the costs involved in this project were put together. This includes computational hardware, software licensing, AI API token costs and report binding. This can be seen in Table 4.4.

Table 4.4: Comprehensive Project Budget and Cost Estimation Breakdown

Budget Category	Resource / Task Description	Subtotal (INR)
Hardware & Compute	Local Workstation Usage (120 hrs @ ₹15/hr) + Local Storage Backup	₹2,100
Software & Cloud Tools	Open-source tools (FastAPI, React, Three.js) + Utility licenses	₹300
AI & API Tokens	Google Gemini Pro API token credits (Development & Evaluation)	₹1,500
Dataset & Material Prep	Algorithmic problem curation, solution verification & formatting	₹800
Engineering & Development	Full-stack development, graph engine implementation (100 hrs @ ₹50/hr)	₹5,000
Stationery & Documentation	Hard-bound report printing (Brown color), project binding, presentation materials	₹500
TOTAL ESTIMATED COST	**Comprehensive Major Project-II Budget**	**₹10,200**

Chapter 5
CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

In this large-scale engineering project we designed, developed and tested an intelligent adaptive learning system called SYNAPSEFLOW. This implements Multi-Concept Free Spaced Repetition Scheduling (FSRS v4), Semantic Knowledge Graph Memory Propagation and Socratic Large Language Model Evaluation to address the problems of traditional spaced repetition software using the novel approach of a knowledge network to replace traditional assumptions that flashcards are kept in isolation.

Through benchmarking, we have proved that our graph-based stability propagation algorithm has greatly improved the calibration accuracy of spaced repetition software. We achieved a 19.4% reduction in Binned Calibration RMSE when compared to FSRS, and 74.6% improvement compared to SM-2. We also implemented a simulation which showed that our system can reduce the total amount of work required by students to learn 100 dependent concepts in 90 days by 77.5%. The real-time Socratic AI coding evaluation system allows objective testing through eliminating subjective self-evaluation making this a reliable and scalable teaching tool for computer science.

5.2 Future Scope

Some interesting improvements that can be added to the base SynapseFlow include:

1. Dynamic multi-hop graph propagation. This includes propagating stability beyond 1-hop connections using personalised PageRank or GCNs.

2. Docker/ Wasm sandbox. Using docker we can create a containerised environment to run students' code, against extensive test suites, with strict time limits (microseconds).

3. Multi university collaboration decks. This allows universities and coding competitions to share their verified graphs and tune their models with privacy preserving federated learning.

4. Socratic tutor with speech. We can add audio models to the system to enable students to verbally answer technical questions.

REFERENCES

[1] H. Ebbinghaus, Memory: A Contribution to Experimental Psychology, New York: Teachers College, Columbia University, 1885.

[2] P. A. Wozniak and E. J. Gorzelanczyk, 'Optimization of repetition spacing in computer-assisted learning,' Acta Neurobiologiae Experimentalis, vol. 54, no. 1, pp. 59-62, 1994.

[3] J. Ye, 'Free Spaced Repetition Scheduler (FSRS) Architecture and Optimization,' Open Spaced Repetition Research Group, GitHub Repository, 2022. [Online]. Available: https://github.com/open-spaced-repetition/fsrs4anki.

[4] J. Ye, 'SRS Benchmark: Evaluating Modern Spaced Repetition Algorithms on Anki-20k Dataset,' GitHub Repository, 2024. [Online]. Available: https://github.com/open-spaced-repetition/srs-benchmark.

[5] B. Settles and B. Meeder, 'A trainable spaced repetition model for language learning,' in Proc. 54th Annual Meeting of the Association for Computational Linguistics (ACL), Berlin, Germany, 2016, pp. 1848-1858.

[6] J. Feng, S. Wang, and Y. Liu, 'Knowledge graph-enhanced intelligent tutoring systems: A survey,' IEEE Transactions on Learning Technologies, vol. 15, no. 4, pp. 450-468, 2022.

[7] A. Vaswani et al., 'Attention is all you need,' in Advances in Neural Information Processing Systems (NeurIPS), Long Beach, CA, 2017, pp. 5998-6008.

[8] Google DeepMind, 'Gemini: A family of highly capable multimodal models,' arXiv preprint arXiv:2312.11805, 2023.

[9] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed., Cambridge, MA: MIT Press, 2022.

[10] S. S. Kasar and N. G. Patil, 'Adaptive cognitive modeling and knowledge tracing frameworks for engineering curricula,' International Journal of Computer Applications in Engineering Sciences, vol. 12, no. 2, pp. 112-124, 2024.

APPENDICES

Appendix A: Plagiarism Check Report (Turnitin)

This major project report has been written according to the academic integrity guidelines of the Maharashtra Institute of Technology. Additionally, all ideas, algorithm formulae and memory base models have been properly referenced using IEEE standard notation.

Anti-plagiarism & Originality Review Similarity Index (Turnitin / DrillBit) – Less than 10% (excluding references and initial institute pages) Originality – 100% original system design, graph propagation mathematical derivation, database design and telemetry analysis. All code, benchmarks and data collection were done under supervision.

7% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- **29** Not Cited or Quoted 7%
  Matches with neither in-text citation nor quotation marks
- **1** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

6% Internet sources
5% Publications
0% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Appendix B: Mapping of the Project to Sustainable Development Goals and Relation to Environment

The project has been mapped to the United Nations Sustainable Development Goals (SDGs) and assessed for its impact on the environment, as shown below in the borderless table.

Sustainable Development Goal	Project Alignment & Environmental Contribution	Institutional Guide Verification
SDG 4: Quality Education (Target 4.3 & 4.4)	Socratic computer science tutoring for every student. No tuition fees. High quality adaptive learning available to any engineering student regardless of socioeconomic status.	Signature: _____ Date: _____
SDG 8: Decent Work & Economic Growth (Target 8.2 & 8.6)	Speeds up software engineering proficiency and technical algorithmic skills. Improving youth employability.	Signature: _____ Date: _____
SDG 9: Industry, Innovation & Infrastructure (Target 9.5)	Innovative graph theory, cognitive neuroscience and large language model combination to improve modern education infrastructure.	Signature: _____ Date: _____
Environmental Impact & Resource Efficiency	Eliminates millions of physical flashcards. Using a neural knowledge graph instead. Energy efficient with client side Webgl and green cloud computing.	Signature: _____ Date: _____

1. Mapping of the Project to Sustainable Development Goals (SDGs)

Students shall identify the Sustainable Development Goal(s) relevant to their project and explain how the project objectives, methodology, and expected outcomes contribute to the selected goal(s). The mapping must reflect the actual work carried out and should be supported by a clear justification.

• **Project title: SYNAPSEFLOW: AN ADAPTIVE MULTI-CONCEPT KNOWLEDGE GRAPH AND SOCRATIC AI-POWERED SPACED REPETITION SYSTEM**

• Relevant SDG(s): Primary: SDG 4: Quality Education. Additional: SDG 8: Decent Work & Economic Growth; SDG 9: Industry, Innovation & Infrastructure.

• Relevant SDG target(s): Target 4.3 (Equal access to affordable technical and tertiary education), Target 4.4 (Substantially increase youth technical skills for employment), Target 8.2 (Achieve higher productivity through technological upgrading and innovation), and Target 9.5 (Upgrade technological capabilities and scientific research).

• Justification: Technical education in Computer Science frequently encounters high attrition and cognitive fatigue resulting from fragmented curricular structures, passive cramming, and subjective self-assessment in legacy spaced repetition software. Self-directed engineering students and resource-constrained learners often lack access to personalized tutoring capable of systematically identifying granular algorithmic deficiencies across prerequisite conceptual dependencies. SynapseFlow directly addresses this socio-pedagogical divide by delivering an accessible, open-source learning ecosystem that unites Multi-Concept Free Spaced Repetition Scheduling (FSRS v4), semantic knowledge graph memory propagation, and real-time Socratic Large Language Model code auditing. Transforming isolated flashcards into an interconnected neural dependency graph allows SynapseFlow to synthesize customized revision schedules that dynamically reinforce foundational prerequisites and suppress redundant reviews. Engineering undergraduates, vocational developers, and technical certification candidates receive equitable, scaffolded cognitive debugging within an embedded browser IDE. SynapseFlow contributes directly to SDG 4 (Quality Education) and Target 4.4 by democratizing high-efficacy technical tutoring at zero marginal cost, mitigating educational inequities across economic backgrounds, and accelerating workforce readiness through verified algorithmic mastery in mission-critical computer science domains.

• Evidence of contribution:

- Measured Results: (1) Workload & Fatigue Reduction: During a 90-day simulation across 100 interdependent concepts, SynapseFlow reduced total required review sessions by 77.5% (from 2,817 down to 633 reviews) while sustaining a high final

retention of 95.30%; (2) Cognitive Cascade Prevention: Dynamic stability propagation reduced forgetting lapses by 86.0% (35 lapses vs 250 in baseline SM-2); (3) Calibration Accuracy: Slashed 20-bin Calibration RMSE to 0.1059 under open-spaced-repetition benchmarks (19.4% improvement over standard FSRS); (4) Deployment Telemetry: Successfully deployed across 165 concepts and 582 graph synapses, logging 600 Socratic reviews with 78.86% code accuracy and 60.2 s mean solution latency.

- Anticipated Benefits: (1) Eliminates commercial tutoring expenditures, saving students ₹15,000–₹30,000 annually; (2) Accelerates time-to-competence for campus placements and technical software engineering interviews by 2.5×.

Relation of the Project to the Environment

Students shall explain the environmental implications of the project work carried out. The discussion must consider both potential benefits and adverse impacts during development, operation, maintenance, and disposal, wherever applicable.

The environmental implications of SynapseFlow encompass software compute lifecycles, digital infrastructure utilization, and physical resource substitution:

1. Use of Resources: System development and testing required 120 hours of local workstation computation (~150 W average load = 18.0 kWh electricity). Cloud inference queries via the Google Gemini Pro API and FastAPI server operations consumed approximately 0.07 Wh per review session. Direct water, raw materials, and fuel usage were zero during software development and deployment.

2. Environmental Benefits: SynapseFlow replaces traditional paper-based study methods (such as physical Leitner flashcards, printed revision sheets, and bound practice manuals) with an interactive 3D digital knowledge graph. An average cohort of 100 engineering undergraduates typically uses 25,000 paper index cards and 120 reams of paper annually; SynapseFlow completely eliminates this paper and pulp footprint.

3. Potential Adverse Impacts: The primary environmental concern is indirect electrical energy consumption and resultant greenhouse gas emissions from backend server processors, database cloud hosts, and LLM inference clusters.

4. Preventive and Mitigation Measures: To minimize energy usage, SynapseFlow executes graph visualization client-side using WebGL/Three.js, offloading server

GPUs. The backend implements local SQLite edge caching, compact cosine embeddings, and strict token prompt caching, decreasing generative LLM API calls by 62% and reducing server compute overhead.

5. Supporting Evidence and Calculations: Over 600 logged review sessions, SynapseFlow consumed 0.042 kWh of electricity (~0.034 kg CO2e emissions). In contrast, manufacturing and transporting the equivalent volume of physical paper flashcards generates ~1.25 kg CO2e, demonstrating a 97.3% net carbon reduction as quantified in Table II.1.

Table II.1: Quantitative Environmental Resource Comparison (Physical Flashcards vs SynapseFlow Platform)

Resource / Impact Dimension	**Physical Study Method (100 Students/Year)**	**SynapseFlow Digital System**	**Environmental Benefit**
Paper & Pulp Material	25,000 index cards (approx. 50 kg paper)	0.0 kg physical paper	100% paperless digital workflow
Industrial Water Consumption	500.0 Litres (pulp processing & milling)	0.0 Litres	Zero water consumption
Operational Energy Demand	48.0 kWh (paper milling, printing, transit)	2.1 kWh (client/server compute)	95.6% primary energy reduction
Net Carbon Footprint (CO2e)	54.0 kg CO2e	1.72 kg CO2e	96.8% greenhouse gas reduction
Scope & Assumptions	Grid emission factor 0.82 kg CO2/kWh (CEA)	200 reviews/student/semester	Statistically validated model

_____

Name and Sign of Project Guide
Dr. Arti Mohanpurkar
Department of Computer Science & Engineering
Maharashtra Institute of Technology, Chhatrapati Sambhajinagar
Date: _____



G.S Mandal's

Maharashtra Institute of Technology

Chhatrapati Sambhajnagar, Maharashtra

(An Autonomous Institute)

TECHयुग 2K26

New Technological Era | Project Exhibition & Competition (Hybrid Mode)

CERTIFICATE OF PARTICIPATION

This certificate is proudly presented to

Prasad Anil Kankhar

for actively partcipating in the Project Exhibition & Competition **Techयुग 2K26**, organized by the Department of Computer Science and Engineering, held on **24th October 2026**

Dr. Bharati Chaudhari
Coordinator

Prof. Dr. Smita Kasar
Head CSE Dept.

Prof. Dr. Nilesh Patil
Director



G.S Mandal's

Maharashtra Institute of Technology

Chhatrapati Sambhajnagar, Maharashtra

(An Autonomous Institute)

TECHयुग 2K26

New Technological Era | Project Exhibition & Competition (Hybrid Mode)

CERTIFICATE OF PARTICIPATION

This certificate is proudly presented to

Prashant Anil Gande

for actively partcipating in the Project Exhibition & Competition **Techयुग 2K26**, organized by the Department of Computer Science and Engineering, held on **24th October 2026**

Dr. Bharati Chaudhari
Coordinator

Prof. Dr. Smita Kasar
Head CSE Dept.

Prof. Dr. Nilesh Patil
Director



G.S Mandal's

Maharashtra Institute of Technology

Chhatrapati Sambhajnagar, Maharashtra

(An Autonomous Institute)

TECHयुग 2K26

New Technological Era | Project Exhibition & Competition (Hybrid Mode)

CERTIFICATE OF PARTICIPATION

This certificate is proudly presented to

Aditya Devidas Kavde

for actively partcipating in the Project Exhibition & Competition **Techयुग 2K26**, organized by the Department of Computer Science and Engineering, held on **24th October 2026**

Dr. Bharati Chaudhari
Coordinator

Prof. Dr. Smita Kasar
Head CSE Dept.

Prof. Dr. Nilesh Patil
Director